Proven C Book

Modern C, built on the proven library

draft — translation in progress

v0.2.0 · last updated 2026-08-05

rubidus

rubidus@gmail.com github.com/rubidus-api/proven_c_book

An introduction to C, and an introduction to the proven C library.

Written for readers who are just starting out with C.

Contents

Preface

This book is an introduction to proven, a C library I wrote, and it is also a piece of promotion for it.

But before you can recommend a tool, you have to show why it is needed. The problems proven addresses live in the subtle corners of C — the limits of representation, the rules of conversion, the lifetime of storage, the areas the standard declines to promise anything about. To someone who has not seen those corners, proven looks like an unnecessary contraption. So I decided to explain the problems first, and the library afterwards. That is why most of this book is written in plain standard C, and why proven does not appear until Part XII. Having read the earlier chapters, you may well decide you do not need proven. That is a fine outcome too.

There is also, in these pages, a certain amount of affection for C. Explaining why a language past its fiftieth year is still here, and what its design gave up in order to gain what it gained, tends to produce that. I have not hidden it.

Every piece of code in this book has actually been compiled and run. The printed output is not transcribed by hand: it is what the machine produced when the examples were run during the build, cross-checked with two compilers (gcc and clang).

One more disclosure. **This book was written with AI as an assisting tool.** What to cover and in what order, which principles the exposition follows, what to include and what to leave out — all of that I decided; the manuscript was written to those instructions and then reviewed by me. Technical claims were checked against the standard and primary sources, and, as said above, the examples are verified by a machine that actually runs them on every build — which means that whether a human or an AI wrote it, **code that does not run does not appear in this book**.

Errors will remain nonetheless. Those are mine, not the tool's. If you find one, please tell me.

Copyright and contact

Contact is by email. Error reports and correction requests are better made through the GitHub repository — they leave a record, and other readers can see them.

What I accept is **error reports only.** Incorrect statements, examples that do not match, typos, stale information — tell me and I will gratefully fix them. Manuscript contributions, on the other hand — a new chapter or section written and sent in — I do not accept. The structure and voice of this book are better kept under one person's judgement, and I would rather keep the copyright situation simple.

rubidus@gmail.com
github.com/rubidus-api/proven_c_book

A note on this translation

The Korean edition is the original and is complete — 13 parts, 69 chapters, appendices A–E and an index, about 287 pages. This English edition is translated from it chapter by chapter, and **chapter numbers are kept identical to the original**, so a cross-reference to "chapter 49" means the same chapter in both editions. Chapters not yet translated are listed where they belong, and the parts they sit in are marked accordingly.

The two editions are kept in step mechanically: `scripts/sync-status.py` records the hash of the Korean source each translated file was made from, and every build reports any chapter whose original has changed since. Per-chapter status is in `TRANSLATION.md` in the repository.

Until a chapter arrives here, the complete text is the Korean edition:

- Web — rubidus-api.github.io/proven_c_book/ko/
- PDF — the `ko` asset of the current release

Part I — Ground

1 Setting the scene

By the end of this chapter

What programming is, what kind of language C is and where it currently stands, and why a new introduction to C is needed right now. This chapter also settles how the book was written and how it is meant to be read.

1.1 What is programming

A computer is a machine that does what it is told. The difficulty is in the telling. A computer does not understand human speech; it moves only when it receives brutally simple instructions in order. **Programming** is the work of turning what we want into a list of such simple instructions, and the agreed notation for writing that list down is a **programming language**.

There are hundreds of languages. Some sit close to the human side and are comfortable to write; others sit close to the machine and let you handle it down to its details. C, the subject of this book, is the leading example of the latter — the language of systems, which has held the machine-side seat for half a century.

1.2 Where C is

C is not easy to spot. Other languages stand at the front of flashy apps and websites. Lift the faces of those, though, and C is nearly everywhere underneath.

◉ **A day spent on top of C**

The alarm goes off on your phone in the morning — the core of that operating system (the kernel) is written in C. You send a message and the firmware of the radio module goes to work — C. You open the web and a server and a browser exchange data — the encryption library and transfer tool doing it (OpenSSL, curl) are C. The app stores something — SQLite, the most widely deployed database in the world, is C. The braking system in a car, the control panel of a washing machine, a pacemaker: most embedded devices are C. Even when you run machine learning from Python, a large part of the engine actually doing the arithmetic underneath is C (and its neighbours). Learning C means becoming able to read and write this invisible layer.

1.3 The world of C is in motion right now

So why a new introduction **now**? If the language is old, surely an old book will do. It will not — the landscape around C looks quiet but is shifting a great deal, and that shifting is where this book starts.

First, several eras of C coexist inside the language. C is governed by an official specification, the standard, and it has come through C89 (1989), C99, C11 and C17 to the current C23. The trouble is that old standards never retire. Industry is still full of code written exactly as it was in the C89 days, many textbooks are written in that era's idiom, and alongside them C23 has brought new vocabulary such as `bool` and `nullptr` into the standard. Dialects thirty years apart live together under the single name "C".

Q If there is that much old-standard code about, is it not more practical to learn the old idiom first?

A Separate reading from writing and the answer settles. The day will come when you have to **read** old code — which is why this book covers history and the stories behind old idioms throughout. But there is no reason for code you **write** today to be in the old idiom. The newer standards are the result of decades of field experience filling in the traps of the older ones. The earlier you are in your learning, the more you gain by starting from today's safest idiom; the old idiom is quite enough as background knowledge of "why they wrote it that way."

Second, neighbouring languages are encroaching on C's territory. C++, which started under the same roof, has gone through revisions at a rapid pace and become an enormous language — vastly expressive, but grown large enough that people say hardly anyone knows all of it. From another direction a new generation of systems languages has appeared. Rust promises to seal off, at the language level, the memory accidents C has long suffered; Zig marches under the banner of "succeed C, but simpler and more honest." Each is pushing into ground C used to hold alone. Rust code beginning to enter operating system kernels was a symbolic moment.

Third, under that pressure C itself is showing signs of change. C23 was the largest revision in recent memory, and the committee continues work on tightening the rules for pointers (provenance) and on the safety discussion. C is a slow-moving language, but the direction is clear — towards clearer contracts and safer defaults.

⚠ "C is an old language, so learning it will soon be pointless"

It is a plausible worry — the new languages are arriving in force. But the actual numbers point the other way. As we saw above, the world's infrastructure stands on C, and this layer is so expensive to replace that it moves only on the scale of decades. Even the new languages are born with the ability to talk to C (a C interface) as a condition of survival — C is the lingua franca of the systems world, so whether you write Rust or Zig you end up having to read and understand C. Far from ageing out, C is the kind of language that grows **more** important as its neighbours multiply.

Q Then would it not be better to start with Rust or Zig instead?

A Honestly: it depends on your goal. If the goal is to build one safe application right now, other choices are good too. But if the goal is to understand **how a computer actually works** from the ground up, there is no better teaching material than C. C is the language that wraps the machine most thinly, so learning C turns out to be the same thing as learning about the machine, memory and the compiler. And that understanding is capital you can carry with you into Rust or Zig unchanged. This book is here to supply that capital in today's idiom.

1.4 Why this book was made

That shifting landscape is the first reason. With several standards coexisting, neighbouring languages competing, and C itself changing, what is needed now is not an introduction written by the inertia of the old idiom but one that **starts from today's C (C23) and today's practice**. So the table of contents was rebuilt from scratch. Instead of listing grammatical items in order, the book starts with the basics of the machine and meets C in the order in which the concepts become necessary.

The second reason is best stated plainly. The author builds and publishes a C library called **proven**. proven aims to provide, in verified form, the fundamentals where C programming most often goes wrong — handling strings, processing input, converting numbers. Most of C's famous traps go off in this layer of fundamentals, and the standard library, for historical reasons, carries those traps as they are. The later parts of this book show why the traps are genuinely dangerous and then use proven as the basis of examples for how they can be blocked. This book is also meant to make that library known — but the order is respected. First the concepts are learned properly in standard C alone; proven appears at the point where its necessity has proved itself.

Q If it is a book promoting a library, does the content not lean that way?

A The structure is built to prevent it. The first thirty-odd chapters do not use a single line of proven — they proceed purely on the basics of computing and standard C. proven appears only after the text itself has made plain "why a tool like this is needed", and even then the standard approach is shown first and proven offered as a comparison. The C knowledge in this book stands entirely on its own without proven.

1.5 How to read this book

This is a book to be read. That sounds obvious, but as a promise from a programming book it is unusual.

It was written to be readable straight through. So there are no exercises and no instructions to "try it yourself." You do not need to be sitting at a computer — on a sofa, on the train, you can read it front to back like a novel. Every piece of code is demonstrated on the page from beginning to end, and the results of running it are printed on the page as well. Those results are not decoration but the real thing: every example in this book carries the output obtained by a machine actually compiling and running it.

If you want drills, another book is better. Designing good assignments and walking you through them step by step is something the existing good C primers do far better. This book's place is beside them — the reading side, the one that explains why things are shaped as they are.

It proceeds by question and answer. Throughout the text, the question a reader might have just formed is asked on the spot and answered immediately. It is best to pause for a moment when you meet a question and try to answer it yourself before reading on, but reading straight through is fine too — the exchange is part of the explanation.

It does not ask you to memorise. Anything worth remembering comes back at the head of a later chapter under the name "looking back", where you meet the previous chapter's material again as a slightly deeper question. That is this book's revision device. It is fine to have forgotten — the answer is right next to it.

It moves quickly in small chapters. A chapter is short. It handles one idea and moves on. Large subjects pass by several times across several chapters, spiral-fashion, a little deeper each time. Not everything is said in one chapter, so if you are left curious about something, that is usually deliberate — you will meet that curiosity again a few chapters later.

Now we begin. The first step is not C but the computer itself — why C looks the way it does only makes sense once you have seen the machine it was born on. The next part starts by drawing the computer as a very simple picture.

Part II — How computing works

This part teaches no C syntax at all. Instead it lays the floor that every later chapter will stand on. We climb in three steps.

Machine and memory (chapters 2–4) — the computer drawn as a simple picture of three parts, then a walk down the locker corridor called memory. A tangible story.

The ladder of representation (chapters 5–8) — how to put integers into those lockers, then numbers with a decimal point, then characters, then streams of characters. One rung at a time.

Speed and abstraction (chapters 9–12) — how memory split apart as machines got faster (registers, caches), the machinery of speed, the compiler as an editor, and therefore why C is an "abstract language" after all. This part's conclusion is completed here.

At every step, the history of C walks alongside. The point is not to make you memorise dates and names — it is that history is the shortest explanation of why today's C is shaped as it is.

Not yet translated: 8, 9, 10, 11, 12
(read these in the Korean edition)

2 A simple model of the machine - the birth of C

By the end of this chapter

You will be able to draw a computer as a simple picture of three parts — a place that calculates, a place that remembers, and something that keeps time. You will learn what a bit, the smallest unit of information, is and why base two of all things. And you will see how a language called C was born in the days of that simple machine.

Looking back Chapter 1 said that C is still alive inside operating systems, embedded devices and the infrastructure of the internet. How is a language past fifty still in that position?

A Because the position C occupies is a special one. It is the lowest layer at which machine and human meet, and while machines have got faster over half a century, the skeleton of how they work is remarkably unchanged. This chapter looks straight at that skeleton. As long as the skeleton stays, the language of the skeleton stays too.

2.1 Drawing a computer as three parts

A computer is a complicated object. But the first step in understanding anything complicated is always to draw a boldly simple picture. From here we draw the computer as exactly three parts.

First, the **CPU**. The place that calculates. It adds, compares, and decides what to do next.

Second, **memory**. The place that remembers. The material for the calculation and the result of the calculation both sit here.

Third, the **clock**. The thing that keeps time. It ticks at a steady interval like a metronome, and the CPU works one step per tick.

In this picture, what the computer does is this and nothing else: **on every tick of the clock, the CPU fetches one thing to do from memory and does it.** Endlessly repeated.

Q Is the gigahertz (GHz) in advertising copy — "four billion times a second" — this clock?

A It is. 4 GHz means the metronome ticks four billion times a second. The CPU advances one step per tick, so the faster the beat, the more steps in the same time. That said, the number of steps is not the whole story, as chapter 10 will show — doing several things in one step is the real weapon of a modern CPU.

Q How can games, video and artificial intelligence come out of nothing but "fetch it and do it", repeated?

A In the same way that a brick is simple but you can build a castle out of bricks. Even simple steps, taken billions of times a second and organised in layers — small jobs into functions, functions into programs, programs into an operating system — can

build anything however complicated. This "stacking in layers" is **abstraction**, the heart of computing, and this whole book is the story of climbing that staircase.

What the CPU fetches as "the next thing to do" is called an **instruction**. An instruction is nothing grand. "Add this number to that one", "put this value in that slot", "if the last result was zero, jump somewhere else" — brutally simple directions at about that level. A program is a list of such instructions in order, and programming is the work of writing that list in a way a human can bear.

2.2 The bit — the smallest unit of information

Memory, we said, is the place that remembers. What does it remember, and in what shape?

What an electronic circuit does best is distinguish two states. The voltage is high, or it is low. The switch is on, or it is off. The smallest unit of storage, holding one of those two, is called a **bit**. We give the two states the names 0 and 1.

> ⚠ **"Somewhere inside the computer, 0s and 1s are written down like letters"**
>
> It is a plausible thought — every book and every screen paints computers as full of ones and zeros. But there are no digit-shaped characters inside the machine. What is actually there is nothing but physical state: a high or low voltage, a charge present or absent. 0 and 1 are **names** we attach to those physical states so we can read them — an interpretation. This distinction is not idle pedantry. Separating "what is stored" from "how it is to be read" is a theme this book returns to again and again, and it is why the same string of bits can later be read as a number or as a character (chapters 6 and 7).

Q Why two states in particular? Would it not be far more efficient to put ten states, 0 through 9, into one switch?

A Decimal computers were in fact built early on. But distinguishing ten voltage levels means narrowing the gaps between them, and narrow gaps slip into a neighbouring level under noise or a change in temperature — that is, they are easy to get wrong. Two states give the widest possible gap, so the circuits are cheap, fast and robust against noise. Binary won not for mathematical elegance but for **engineering honesty**.

One bit distinguishes two things. Bundle several bits and the number of distinguishable cases grows multiplicatively. Two bits give four (00, 01, 10, 11), three give eight. A bundle of eight has a name — the **byte** — and one byte distinguishes 256 cases.

Q Why eight, of all bundle sizes? Was it eight from the start, and everywhere?

A No — the size of a byte has never been fixed at 8 bits. Originally "byte" meant **the bundle that holds one character**, and its size differed from machine to machine. Early computers really did have 6-bit, 7-bit and 9-bit bytes, and some machines had 36-bit words that they cut into six 6-bit bytes. Eight became dominant when IBM's mainframes of the 1960s (System/360) adopted it and the industry followed. That is why communication

specifications, where precision is everything, still say **octet** (exactly 8 bits) instead of byte — a trace of the history in which the word "byte" alone guaranteed no size.

And this is not only an old story. Some signal-processing chips (DSPs) still have a smallest storage unit of 16 or 32 bits, so in the C running on them a byte is 16 or 32 bits. On machines you are likely to meet it is 8 bits essentially everywhere, but "a byte is always 8 bits" is custom, not promise.

The definition in the C standard (C23) is correspondingly careful. In C23 a **byte** is "the smallest addressable unit of storage that can hold one character of the basic character set", and the number of bits in it is published under the name `CHAR_BIT` — pinned down only as **at least 8**, not exactly 8. This book, following the common machines, will proceed with byte = 8 bits, but records here that this is the practice rather than the standard's guarantee. (The committee is discussing fixing it at 8 outright in a future standard — half a century of custom being promoted to a promise.)

∑ Bundle size and number of cases

The number of states a bundle of n bits can distinguish is, since each position independently takes two values,

$$2 \times 2 \times \cdots \times 2 = 2^n$$

So $2^8 = 256$, $2^{16} = 65536$, $2^{32} \approx 4.3 \times 10^9$. 2^n keeps appearing in this book — the size of storage, the range of representable numbers and the count of addresses are all of this form.

Representing a **number** with a bundle works on the same principle as the decimal notation we use: positional notation. Just as the decimal number $304 = 3 \cdot 10^2 + 0 \cdot 10^1 + 4 \cdot 10^0$, the binary number $101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$. Each position simply carries a power of 2 instead of a power of 10.

Q How do we handle large numbers with a byte that only has 256 cases?

A Just as in decimal you add digits when one is not enough, bytes are joined together to hold larger numbers. How many it is natural to join is fixed per machine — and that "natural bundle" is the word, the subject of the next chapter.

2.3 C was born in the days of that simple machine

The three-part picture drawn so far is far too simple for today's computers (that is the story of chapters 9 and 10). But for a computer of the early 1970s the picture was **very nearly the literal truth**. The clock ticked a few hundred thousand times a second, the CPU really did take one step per tick, and memory really was a row of lockers.

C was born in exactly those days, on exactly that simple machine — not out of nothing, though, but at the end of a short and distinct lineage. Knowing that lineage explains several features of C's appearance at a stroke, so it is summarised here.

The starting point was an ambitious British language of the 1960s called **CPL** — so ambitious that machines of the day struggled to implement it, until Martin Richards at Cambridge distilled its essence into a small, practical language called **BCPL** (1967). BCPL's radical simplicity was this: **there is only one type.** Every value is just one word-sized slot, and whether to use it as a number or as an address is decided by the operation. It pushed "what is in the slot is only bits, and interpretation decides" — the lesson of chapter 3 — across the entire language.

At Bell Labs, Ken Thompson, building early Unix on an ageing PDP-7, made **B** (around 1969), BCPL trimmed further to his taste — still one type, and the source of terse idioms that survive today (`++` among them). Then the labs brought in a new machine, the PDP-11, and B's premise broke. The PDP-11 addressed memory in **bytes** rather than words (chapter 3's corridor is exactly this shape), and data of different sizes — one-byte characters, two-byte integers, and floating point — had to be handled distinctly. A language in which "everything is one word" could not drive this machine properly.

So Dennis Ritchie brought **types** into B — distinguishing characters from integers, creating pointers that know what they point at, and adding structures. Those modifications accumulated between 1971 and 1973 and earned a new name: C. In other words, C's type system was not a philosophical design but a response to **the reality of a byte-addressed machine**. That C is byte-oriented to the bone (the byte of chapter 2, the corridor of chapter 3), that pointers sit at the centre of the language, and that its typing is nonetheless on the loose side (an inheritance from a one-type ancestor) — all are traces of that lineage.

Two footnotes. Idioms that look familiar today were not there from the start — today's `+=`, for instance, was written `=+` in early C and only took its current shape some years later (out of confusion: was `x=-1` a subtraction or an assignment?). Languages are refined by eating the injuries of real use. And in this era C had no standard and no specification document — the definition of the language was effectively "whatever Ritchie's compiler accepts." What commotion that looseness grew into, and how it summoned a standard (C89), is picked up in chapter 10.

So early C was, with only slight exaggeration, **an honest nickname for the machines of its day**. One C operation corresponded to one or two machine instructions, and a programmer writing C could see clearly what the machine was going to do. C's reputation as "a language close to the machine" comes from here — and the story of how that reputation became only half true is in the later chapters of this part (9–12).

◉ The portability revolution — rewriting Unix in C

Around 1973 the heart of Unix was rewritten from assembly into C. It was radical at the time — the common sense being that an operating system had to be written at the machine-code level. The effect was overwhelming. Assembly Unix was tied to the PDP-11, but Unix written in C could be moved to another machine "as long as there is a C compiler." An operating system had stopped being the property of a particular machine. Today the kernels of Linux, Windows and macOS are all written in C (or its descendants), and the tradition that a new processor gets a C compiler ported to it first began here.

Q Is there no loss in learning, today, a language fitted to a machine of fifty years ago?

A Quite the opposite. The skeleton of the machine — memory, addresses, bytes, the sequential execution of instructions — has not changed in fifty years, and C remains the language wrapped most thinly around that skeleton. Learning C is therefore less about memorising one language's grammar than about learning the computer itself. Whatever language you use afterwards, C is almost always underneath it.

To summarise this chapter's picture: the machine repeatedly fetches an instruction from memory on the beat of the clock and executes it, and memory is made of bits, the two-state minimum unit. C was born when this picture was nearly the truth, by translating this picture into a language.

The next chapter looks closely at memory, one of the three parts. We walk the locker corridor and read the number attached to each slot — the **address** — and measure the natural bundle the machine picks up at once — the **word** — and inspect the two orders in which a multi-byte number can be put into the lockers — **endianness**. C's famous notion of the pointer was born in this corridor.

3 Words and addresses - the archetype of C

By the end of this chapter

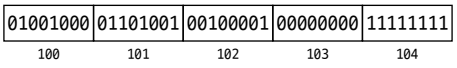
You will be able to draw memory as a long corridor of numbered lockers. That number is the address, and how many slots the machine handles at once is the word. You will learn to tell apart, in pictures, the two orders for storing a number that spans several slots — little-endian and big-endian.

Looking back Chapter 2 said that a single byte distinguishes only 256 cases, and that larger numbers are held by joining several bytes together. So how does the machine know that the joined bytes are "one lump"?

A It does not — and that is where this chapter starts. Memory itself records nothing about which slot belongs with which. What knows about the lump is not the memory but **the side doing the reading**. The program simply decides "from here I shall read four slots as one number." Chapter 2's misconception — the separation of what is stored from how it is read — gets its first real workout here.

3.1 The locker corridor

Drawn as a picture, memory looks like this. Along a long corridor, lockers of identical size run in an endless row, and every locker carries a number. One locker holds 1 byte. The numbers start at 0 and go up by one. That number is the **address**.



What is inside a slot is always just eight bits. Whether those eight bits are a number, a character, or a fragment of something larger is known neither to the slot nor to the corridor — the interpretation of the reader decides.

There is one important fact about addresses that is not visible at first glance. **An address is itself a number.** A locker number is just an integer, and being an integer, it can be put inside another locker. "In slot 347, write the number 512" — nothing strange about it. This ordinary idea later becomes C's most famous concept, the **pointer** (chapter 30). For now it is enough to take away "an address is a number too, so it can be written down anywhere."

Q How long is the corridor — how many lockers are there?

A The number of bits used to write an address decides. If an address is an n-bit number, at most 2^n lockers can be distinguished (chapter 2's 2^n is back already). In the days of 32-bit addresses the corridor was at most about 4.3 billion slots — 4 GiB — and today's 64-bit addresses make a corridor longer than anyone can practically fill. This is what phrases like "the 4 GB memory limit on 32-bit systems" really refer to.

3.2 The word — the machine's natural handful

Lockers come one slot at a time, but if the machine picked up one slot at a time as it worked it would be far too slow. So a CPU is built to grab several slots in **one handful**. The size of that handful — the number of bits the machine handles at once, most naturally — is called the **word**.

This is exactly what "a 64-bit computer" means: that machine's word is 64 bits, that is, 8 bytes. The temporary holders inside the CPU (registers) are that size, the passage to and from memory (the bus) is matched to that width, and addresses are usually handled as numbers of that size too. You might call the word the machine's "hand size" — a machine with a big hand grabs a bigger number at once.

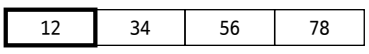
Q So on a 64-bit machine, is everything stored in 8-byte units?

A No. Storage is still free at byte granularity — a one-byte character and a four-byte number live in the same corridor. The word is "the size the machine picks up most comfortably", not "the size of everything." C really does have several number types of different sizes (chapter 22), and the question of where it is convenient to place data of a given size comes back in chapter 4 (alignment).

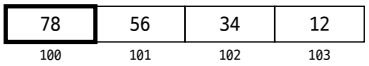
3.3 Endianness — two orders for putting a number in several slots

Now the highlight of the chapter. A four-byte number has to go into four lockers. Say the number splits, in hexadecimal, into the four byte-pieces 12 34 56 78 (two hex digits are one byte). If it starts at slot 100 — which piece goes into slot 100?

There are two schools. **Big-endian** puts the big end first: exactly the order a human writes a number on paper.



Little-endian puts the little end first. It looks reversed, but it is an orderly rule in its own right: "the i-th slot holds the i-th least significant piece."



The computer or phone you are reading this on is almost certainly little-endian (Intel, AMD, and most ARM deployments). It is precisely the situation of date notation — some countries write 2026-08-04 and others 04-08-2026, neither is wrong, but **reading each other's letters verbatim causes accidents**.

> **Q** Big-endian looks natural to the human eye, so why did most machines choose the "reversed" little-endian? What is the advantage?

> **A** Two practical ones.
>
> First, **the starting slot does not move.** In little-endian the least significant piece of a number is always in the slot at the starting address. Whether you read the number above as "all four bytes" or narrow it to "only the low two bytes" or "only the low byte", the slot you start reading at is the same 100. In big-endian you must shift the starting slot and recompute every time you change the width you read. For a machine that often reads the same value through eyes of different sizes, "the start address is invariant" is a considerable simplification.
>
> Second, **it matches the direction of the arithmetic.** Think of adding by hand: you add from the ones place and carry upward. A machine adding multi-byte numbers likewise processes from the least significant end, and in little-endian that lines up exactly with "in increasing address order." Early small CPUs could start adding as soon as the first byte arrived, and this advantage soaked into early designs (the ancestors of the Intel line) and has carried through to today.
>
> To be fair to big-endian: a dump reads directly to the human eye, and comparing bytes whole from the front agrees with the numeric ordering. So big-endian survives where "humans and machines look at the same place", such as communication protocols. Neither is superior — they simply do different things often.

> **⚠ "Read memory from the front and you see the number in the order it was written"**
>
> Plausible — that is how a number on paper is read. But on a little-endian machine the number 12 34 56 78 sits in memory in the order 78 56 34 12. Look at memory in slot order (which is exactly how a debugger or file dump shows it) and the number appears "reversed". It is not reversed; the convention for the order of storing is simply different. Remember: a memory dump is not a number written on paper.

◉ NUXI — the ghost born of ordering

Engineers porting early Unix to another company's computer saw something odd on screen. Where UNIX should have been printed, NUXI appeared. The two machines differed in endianness, so the order of the characters within each two-byte bundle came out wholly reversed. The episode became known as "the NUXI problem", the byword for endianness accidents. The same class of accident still happens — saving a file on one machine and reading it on another, or passing numbers over a network, breaks the numbers unless the endianness is agreed. Which is why internet protocols settled on **network byte order**: "send big-endian on the wire."

Q Can I check with a program that my computer is little-endian?

A You can — put a multi-byte number in memory and read just its first slot as a single byte. If the first slot holds the least significant piece, it is little-endian. Actually doing this check in C is the demonstration in chapter 40 (unions) — that is where we get the tool for reading the same storage through different eyes.

3.4 The archetype of C is here

This chapter's corridor picture is also the blueprint of the C language. In the days when C was born (chapter 2), the people designing the language did not hide this shape of the machine but lifted it, as it was, into the concepts of the language. Memory is a sequence of bytes — every piece of data in C has a size in bytes. Every slot has an address — in C you can ask for the address of almost anything. An address is a number too — the type that holds that number (the pointer) sits at the centre of the language. This contrasts with the many languages that seal the "locker corridor" away from the programmer.

This honesty cuts both ways. It is the power to work while seeing right through the machine, and it is the danger that getting lost in the corridor is an accident on the spot. Part VII of this book faces that power and that danger head on.

The next chapter tours the interesting special cases in the corners of this corridor. What is in locker 0, why a two-slot piece of luggage cannot go into just any number (alignment), and even the trick of hiding information in the fact that the last digit of a number is always 0 (tagged pointers) — you will see that the world of addresses is far more like an inhabited neighbourhood than you would expect.

4 Special knowledge about addresses — address 0, alignment, low bits

By the end of this chapter

Three pieces of corner knowledge about the locker corridor. Why locker 0 is special, why a large piece of luggage cannot go into just any number (alignment), and the trick of hiding information in the last digits of a number (tagged pointers). The three different things all called "null" are first told apart here as well.

Looking back Chapter 3 said "an address is a number too, so it can be written down in another locker." But suppose you want to record that there is **no address yet** — how do you indicate "points nowhere"?

A You agree on one special value as the mark for "none." And almost every system chose 0 for that mark — leaving locker 0 empty, used by nobody, and reading "if 0 is written there, it points nowhere." That agreed value is the **null pointer**. But why was locker 0 free to be left empty in the first place — that story is the first section of this chapter.

4.1 What is in locker 0

The answer first: **it differs by machine**. And the difference is interesting.

On desktops, servers and phones, where the operating system runs in protected mode, the OS deliberately makes the area around address 0 a **forbidden zone**. A program that tries to read or write address 0 is caught and terminated on the spot. This is a kindness — mistakenly using the "none" mark (null) as if it were a real address is a very common accident, and because address 0 is forbidden, that mistake does not slip by quietly but is caught loudly right where it happens.

The embedded world is different. On small chips running without an operating system, address 0 is often perfectly good, even important, memory — many microcontrollers place the interrupt vector table (a sort of emergency contact list) near address 0. On such a machine, reading address 0 is legal and meaningful.

Q On a machine where address 0 is real memory, how do you tell the "none" mark from a genuine address 0?

A The distinction does blur, and embedded programmers work conscious of that boundary. More interesting is history's answer — machines really did exist that used a value other than 0 as the "none" mark. The following case is that story.

◉ Machines where the null pointer was not zero

The C standard was careful from the start. That the constant 0 written in source code means a null pointer is the standard's promise, but **what bit pattern** that null pointer actually has inside the machine was left to the machine's freedom. And there were machines that used that freedom. The Prime 50 series used segment 07777, offset 0 as null; the CDC Cyber 180 used the special pattern 0xB00000000000; Lisp-optimised Symbolics machines used a nonzero value, "the address of the NIL object." Look at the bits of a null pointer on such a machine and not one of them is 0 — and yet comparing a pointer against 0 in the source still comes out true, exactly as the standard promises. The 0 in the source is a **symbol**; the representation in the machine is an **implementation** — the "separation of symbol and representation" carried along since chapter 2 repeats here too.

Today's mainstream machines (Intel, AMD, ARM and so on) all use "all bits zero" as null, so you rarely feel this distinction. But in the world of tagged pointers, coming up shortly, the situation of "the value meaning none is not all zeros" is alive and well right now.

4.2 The three nulls — three things alike only in name

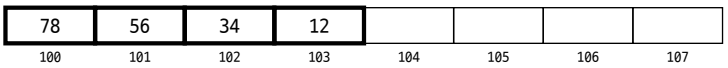
There are three things called "null" around C. Telling them apart once, now, saves great confusion later. They get formal treatment in Part VII (chapter 31); here we merely learn their faces.

- **The null pointer** — what we just saw. The agreed pointer value meaning "points nowhere." It lives in the world of pointers.
- `NULL` — the name (a macro) used to write a null pointer in C source code. The latest C (C23) also brought in a clearer spelling, `nullptr`. It too lives in the world of pointers.
- **The NUL character** — an entirely different object. It is a single **character** of value 0, at position 0 of the character table (chapter 7), written `'\0'` in C. It is one byte of data and is used to mark the end of a string (chapter 34). It lives in the world of characters.

All three have "null" in the name and all three have a 0 tangled up in them somewhere, so they are easy to mix up, but the pointer's null and the character's NUL live in different worlds. Remember them as "strangers who resemble each other only because the value happens to be 0."

4.3 Alignment — a two-slot load cannot go just anywhere

The second piece of corner knowledge about the corridor. Chapter 3 said the machine grabs several slots in one handful (the word). But the machine's hand is not built to take a handful at any position — usually it grabs comfortably only **at numbers that are multiples of its own size**. A four-byte number is grabbed in one go when it sits at an address that is a multiple of 4 (100, 104, 108, ...); an eight-byte one, at a multiple of 8. This rule is **alignment**.



Four bytes starting at 100 (a multiple of 4), as above, are aligned. Put the same number starting at 102 and the alignment is off. What happens then also differs by machine. A tolerant machine (the Intel line) merely **gets slower**, splitting it into two handfuls, but does the job. A strict machine (old SPARC, some ARM eras) raised an error (a bus fault) on the spot and stopped the program. This is one of the classic sources of the portability problem of "code that works only on the machine where it works."

Q Why can the machine not grab anywhere? In the locker metaphor, what gets in the way?

A Because the machine's hand (the bus) sees the corridor as a grid of four-slot cells and moves accordingly. 100–103 is one cell, 104–107 the next. Four bytes starting at 102 straddle two cells, so grabbing them at once means opening both cells and stitching the needed pieces together — work done twice, or refused outright. It is the same as a locker-room rule that two-slot luggage must start at an even number.

4.4 The trick of the low bits — tagged pointers

The alignment rule has an unexpected by-product. Write a multiple of 4 in binary and the last two bits are **always 0**; for a multiple of 8, the last three bits are always 0. That is, the low bits of an aligned address are always zero — so they can be used as **free space to carry information**.

This trick is called a **tagged pointer**. While writing down the address, you tuck a small tag into the trailing digits that are zero anyway. It is genuinely widespread — the runtimes of Lisp-family languages and OCaml distinguish "is this value an integer or an object address?" by a low-bit tag, and JavaScript engines and garbage collectors put status marks on pointers by the same trick. It is not free, of course — the tag must be stripped off (the low bits set back to 0) before the address is actually used.

Here we meet the earlier null story again. In a tagged world, even the special value meaning "none" carries a tag, so **not all of its bits need be 0**. Lisp's "none" (NIL), for instance, is the address of a real special object. Even today, when C's null pointer is uniformly all-zero on mainstream machines, "nonzero none" is still in active service one layer up, in the world of runtimes.

Q May a programmer play the tagged-pointer trick directly?

A It is technically possible in C and is a genuinely used technique, but it is an advanced one with many traps — you need a guarantee of alignment, you must strip the tag before use without fail, and it tangles with the pointer rules to come (provenance in chapter 12, and chapter 32). One conclusion suffices at this stage: the low bits of an address are not "just a number" but a special place that alignment created.

The three pieces of corner knowledge in this chapter — the specialness of address 0, alignment, and the low bits — all come from a single fact. An address is a number, but **not every number is treated alike**. The corridor has a geography.

From the next chapter we turn our eyes to the contents put into the lockers. The first rung on the ladder of representation is the integer — the story of three competing ways to hold negative numbers in bits and C23′s decision, numbers that overflow, and shifting bits wholesale left and right.

5 Representing integers — sign, overflow, shift

By the end of this chapter

The first rung on the ladder of representation. You will see how unsigned integers are circular (modular) numbers, how three ways of holding negative numbers in bits competed until two's complement settled it — and what C23 finally pinned down. Add numbers that overflow and shifting bits wholesale, and the background knowledge about integers is complete.

Looking back Chapter 3 said that several bytes are joined to hold a large number, and we even saw endianness (the order of storing). But every number so far has been zero or above. Negative numbers — where among the bits do you put the minus sign?

A There is nowhere to put it — bits have only 0 and 1, no minus sign (chapter 2). So negative numbers too are made by **agreement**: we decide to **read** certain bit patterns as negative. That there was more than one way to make that agreement, and how the competition ended, is the heart of this chapter.

5.1 Unsigned integers — numbers that go round like a clock

First the world without any worry about minus signs. Read n bits as a plain binary number and you hold 2^n numbers, from 0 to $2^n - 1$ (chapter 2). This is the **unsigned** integer. Eight bits give 0–255.

There is one property of this world you must take with you: **the end joins back to the beginning**. Add 1 to 255 and you get not 256 but — since there is no ninth bit to hold 256 — 0. It is the structure of a clock, where one hour after twelve is one. In mathematics this kind of arithmetic is called **modular arithmetic**.

∑ Modular arithmetic — the exact mathematics of finite numbers

Addition, subtraction and multiplication of n-bit unsigned integers are exactly the operations that take the remainder of the result modulo 2^n:

$$\text{result} = (a + b) \bmod 2^n$$

In eight bits, $250 + 10 = 260 \bmod 256 = 4$. What matters is that this is not "wrong addition" but **a different addition** — a fully defined, predictable piece of mathematics. The C standard likewise defines unsigned overflow not as an error but as this arithmetic.

This circling has a name — **overflow**, or more precisely **wrap-around**. Defined behaviour though it is, it becomes an accident when it happens somewhere you did not expect.

◉ The wall at level 256 — the Pac-Man kill screen

The arcade game Pac-Man has a famous wall. Play proceeds perfectly well up to level 255, and then on level 256 the right half of the screen is buried in meaningless symbols and the game becomes unplayable. The code counted the level number in **eight bits**, so the moment it passed 255 it wrapped to 0, and the routine that drew fruit on the assumption of "what level are we on" drew an absurd number of them and destroyed the screen. The container size the designer thought sufficient — "nobody will get to level 256" — became a wall in front of the best players. A container's size is always a wall that somebody reaches.

5.2 Three agreements for holding negative numbers

Now the negatives. The task is to agree to **read** about half of the n-bit patterns as negative, and historically three agreements were actually used. We compare them by holding -5 in eight bits.

First, sign-magnitude. It imitates human notation directly — use the leading bit as the minus sign (1 means negative) and hold the magnitude in the rest. −5 is `1_0000101`. Intuitive, but it costs two things. `00000000` (+0) and `10000000` (−0) mean there are **two zeros**. And the addition circuit is a headache — adding two numbers of different signs needs a separate procedure of "compare the magnitudes, subtract the smaller from the larger, and decide the sign."

Second, ones' complement. To make a number negative, flip every bit. 5 is `00000101`, so −5 is `11111010`. The addition circuit gets considerably simpler, but there are still two zeros (`00000000` and `11111111`) — and every comparison has to drag along the exception "the two zeros count as equal."

Third, two's complement. Flip the bits and **add one**. −5 is `11111010 + 1 = 11111011`. This rule looks arbitrary at first but is in fact the most elegant — there is only one zero and, above all, **the unsigned addition circuit, used unchanged, gets signed addition right by itself**. No separate procedure, no exceptions.

Q But why the name "complement"? And in "ones' complement" and "two's complement", what do the one and the two refer to?

A A **complement** is "a number that fills something up to a given reference." Decimal makes it easy to feel. For the three-digit number 304, the number that fills each digit up to 9 is 695, called the **nines' complement** (in each position, $9 - \text{that digit}$); the number that fills it up to 1000 is 696, the **ten's complement** ($10^3 - 304$). The relation between them is "nines' complement + 1 = ten's complement."

Do the same thing in binary and the names unravel. The number that fills each position up to **1** — that is, subtracting from `11111111` — is the **ones' complement**, and since $1 - \text{bit}$ in binary is just flipping the bit, that is where the rule "flip them" comes from. And subtracting from 2^n — binary's "1000…0", a **power of two** — is the **two's complement**. The trick "flip and add one" is exactly the relation "nines' complement + 1 = ten's complement" in decimal.

A word on the English spelling. The usual forms are **one's complement** and **two's complement** (there is no "1s'" or "2s'"). But the computer scientist Knuth argued for a witty distinction — the first is a complement with respect to **the ones in every position**, so the plural possessive **ones' complement** is right, while the second is a complement with respect to the **single** number 2^n, so the singular **two's complement** is right. It sounds like grammatical pedantry, but it captures exactly the difference in the two mathematical definitions (per-position reference vs. whole-number reference) — and **the C standard itself adopted the distinction**. Its clause on representations writes the three schemes as "sign and magnitude", "two's complement" and **"ones' complement"**. Knuth's pedantry won in the statute book. In textbook terminology the two's complement is also called the **radix complement** and the ones' complement the **diminished radix complement**.

Σ Why two's complement is elegant — modular arithmetic, reused

The identity of two's complement is the modular arithmetic above. Since the agreement holds $-x$ as the pattern $2^n - x$, in eight bits -5 is $256 - 5 = 251$, that is `11111011`. Then $7 + (-5)$ is, from the circuit's point of view, $7 + 251 = 258 \bmod 256 = 2$ — the answer comes out right by itself. A negative number is merely "counting the other way round the modular clock", so the circuit need not know about signs at all. The only asymmetry is the range — in eight bits it runs from -128 to $+127$, one more negative than positive (-128 has no partner $+128$).

5.3 The competition of the three, and C23's decision

All three schemes were used in real machines — sign-magnitude and ones' complement genuinely existed on early mainframes (the UNIVAC and CDC lines among them). Because such machines were still in service when C was standardised in 1989, the C standard took nobody's side: **it permitted all three representations**. That neutrality was not free — with different representations the result bits of the same operation differ, so the standard had no choice but to leave much of the behaviour of signed integers as "it depends on the machine." Half the reason signed overflow became **undefined behaviour** (chapter 43) lies here.

Meanwhile reality converged on one side. The circuit simplicity of two's complement was overwhelming, so for decades essentially every new CPU used it and the other two schemes went to the museum. And **C23 finally decided — the representation of signed integers is two's complement**. Half a century of practice was promoted to a promise of the standard (exactly the pattern of the "byte = 8 bits" discussion in chapter 2).

Q Now that the representation is pinned to two's complement, is signed overflow defined as wrap-around too?

A No — and this is the subtle, important point. What C23 pinned down is the **representation** (what bit pattern a negative number has), not the **meaning of overflow**. Overflow of signed integers remains undefined behaviour in C23 as well. The reason is optimisation rather than representation — the assumption that "signed numbers do not overflow" is valuable to the compiler (chapter 11) in loop analysis and reordering, so the standard chose to keep it. In summary: unsigned overflow = defined wrap-around, signed overflow = still outside the contract. The practical rules are covered in chapter 23.

⚠ "If an overflow happens, the computer tells you there was an error"

A plausible expectation — it is only decent to be told when something goes wrong. But a CPU's addition circuit merely raises an internal signal (a flag) at the moment of overflow; **by default it neither stops nor notifies the program**. C is the same — unsigned numbers wrap silently, and signed numbers are outside the contract (anything may happen). Pac-Man's screen breaking spectacularly was not an overflow "alarm" but a **downstream accident** caused by the wrapped value. Watching for overflow is the programmer's job, not the machine's — which is also why verified tools like proven check arithmetic later on (chapters 35 and 65).

5.4 Shift — pushing bits wholesale

There is one more basic operation on the bits of an integer — the **shift**, pushing the whole string of bits left or right. After pushing, two questions remain. **Where do the bits pushed out go, and what fills the vacancy?**

Left shift has one answer. Bits pushed off the top are discarded and the vacancy below is filled with 0. Push `00010110` one place left and you get `00101100` — just as adding a 0 on the end multiplies by ten in decimal, one place left in binary is **doubling**.

Right shift has two answers, differing in what fills the vacancy at the top.

- **Logical shift**: fill with 0. This suits unsigned numbers, and one place right is the quotient on division by two.
- **Arithmetic shift**: fill by **copying the sign bit**. A negative number in two's complement has its top bits full of ones (see -5 = `11111011` above), so ones must be shifted in for the meaning "divide by two" to survive. Fill with zeros and a negative number is suddenly read as an enormous positive one.

So CPUs carry two right-shift instructions (logical and arithmetic), and in C the right shift of an unsigned number is logical, while the right shift of a negative signed number was — for a long time "machine-dependent" until essentially every implementation converged on arithmetic in practice. Alongside the settling of two's complement, this is the same direction: practice promoted to promise.

Q Is the shift important enough to justify learning the rules for pushing and filling?

A It is — for two reasons.

First, **because it is the cheapest operation.** For the circuit a shift is about as much work as moving wires sideways, so on nearly every CPU it is among the fastest, single-beat operations. As we just saw, k places left is multiplication by 2^k and k places right is the quotient on division by 2^k — so turning multiplication and division by powers of two into shifts was a classic speed trick. In today's C, though, **you need not play that trick yourself**. Write `x * 2` and `x / 8` in the source, meaning exactly that, and the compiler (the editor of chapter 11) turns them into shifts for you. This is a place where you give up readability and gain nothing.

Second, **because it is the basic move for working in the world of bits.** Packing several values into the bit positions of one integer and taking them back out — assembling UTF-8 bytes as in chapter 7, the tagged pointers of chapter 4, splitting a colour value (RGB), reading the flags of a hardware register — is all a combination of shift and mask: "push to the position you want, and keep only the bits you need." The shift as multiplication has been handed over to the compiler, but the shift as a **placement tool** remains the everyday language of the systems programmer. Its actual use in C's syntax is covered in chapter 24.

Q What happens if you push an eight-bit number eight places, or more? Common sense says everything is pushed out and it becomes 0.

A That very "common sense" differing between machines is the trap. The shift count is processed by a circuit of some width inside the CPU, and machines diverged on what to do when a count at least as large as the width arrived — one family (Intel x86) looks only at the low bits of the count and ignores the rest, so shifting a 32-bit number by 32 leaves it **unchanged**, while others (older ARM and the like) really do push everything out and give **0**. The same code gives different answers on different machines. The C standard's response is by now a familiar pattern — unable to take sides, it put **shifts of at least the width** outside the contract, as undefined behaviour. "Where machines respond differently, the standard gives up on promising" — we meet this pattern formally again in chapter 43.

5.5 Sign extension — from a narrow container to a wide one

The question "what fills the vacancy?" shows up in one more place: moving a number held in an eight-bit container into a sixteen-bit one. What fills the eight new positions at the top?

For unsigned numbers the answer is obvious — **fill with 0** (zero extension). The eight-bit `11111011` (= 251) becomes the sixteen-bit `00000000 11111011` (= 251). The value is unchanged.

For signed numbers the same method causes an accident. The eight-bit `11111011` is -5 in two's complement, but filling the top with zeros gives the sixteen-bit `00000000 11111011` — the leading bit is 0, so it reads as **positive 251**. -5 turned into 251 while changing containers. The correct answer is the same trick as the arithmetic shift — **fill by copying the sign bit**. `11111111 11111011`, still -5. This is **sign extension**.

Q We filled it with a pile of ones and the value is unchanged? Is that a coincidence?

A Not a coincidence but a necessity of modular mathematics. In two's complement the eight-bit -5 was the pattern $2^8 - 5 = 251$, and the sixteen-bit -5 is the pattern $2^{16} - 5 = 65531$. And $65531 = 251 + 255 \cdot 256$ — written in binary, exactly "the original pattern with eight ones laid on top." Copying the sign bit is a trick that performs, with a single bit-copy, the arithmetic of "swapping a complement with respect to 2^8 for a complement with respect to 2^{16}." The elegance of two's complement is at work here too — with sign-magnitude or ones' complement there is no such free extension.

Conversely, **narrowing** from a wide container into a narrow one simply cuts off the upper bits — a cousin of overflow, in that a value that does not fit its container is silently ruined. C has rules for automatically widening small integers before a calculation (integer promotion), and when widening and narrowing happen and what is dangerous about them is treated formally with C's integer types in chapters 23 and 24 — the picture in this chapter (zero fill / sign copy / truncation) is the capital for that.

The background on integers is complete. Unsigned numbers are a modular world that goes round like a clock; negative numbers were settled, after a competition of three agreements, on two's complement, which C23 pinned down; overflow is silent; and shifting and changing containers (extension) only make sense once you know how the vacancy is filled. C's integer **types** and the practical rules are built on this background in chapters 23 and 24.

The next chapter is the next rung on the ladder — beyond integers, the two ways of holding numbers with a decimal point, and the contract called IEEE 754.

6 Representing numbers - the contract called IEEE 754

By the end of this chapter

The next rung on the ladder of representation. You will be able to tell apart the two ways of putting a number with a decimal point into the lockers — fixed point and floating point — and you will meet IEEE 754, the contract that unified a chaos of floating-point formats. One of the most important facts in this book — that a computer's numbers are finite and approximate — takes its place here.

Looking back Chapter 5 got as far as signed integers in bits — including three competing ways of holding negatives, settled on two's complement. But they are all still integers. A number like 1.5 — how is that held in bits?

A The bits themselves have no decimal point. So the decimal point too is made by **agreement** — the same principle by which negative numbers were made by an agreement (two's complement). The way of deciding "where shall we pretend the point is when we read this?" splits into two: pinning the point down (fixed) and carrying the point along in the data (floating). This chapter is the story of those two branches.

6.1 Fixed point — pinning the decimal point down by agreement

The first method is surprisingly simple. **Just use an integer, and change the unit.**

Money is a good example. To store 1,500.25 won, agree that "our ledger is written entirely in units of 0.01 won" and store the integer 150025. The decimal point is stored nowhere — everyone reading and writing merely shares the agreement "pretend there is a point two digits from the end." This is **fixed point**: the position of the point is fixed by agreement.

The virtue of fixed point is that it **is** an integer. Addition and subtraction are as fast and exact as integer operations, with no approximation. So it is still in active service for money calculations and on small embedded chips with no decimal hardware. Its weakness is stiffness — one agreement cannot cover both very large and very small numbers. You cannot write the mass of a galaxy and the mass of an atom in the same ledger of hundredths.

6.2 Floating point — carrying the point in the data

The second method transfers **scientific notation**, learned in science class, to the machine. That is, writing a number as 6.02×10^{23} — split into "the meaningful digits" and "how many times ten was multiplied in." Store both the kernel (the significand) and the multiplication count (the exponent) as data, and the decimal point **floats** along with the exponent — hence **floating point**. The machine uses 2 instead of 10: it holds numbers in the form $\text{significand} \times 2^{\text{exponent}}$.

The power of this method is that it covers **an enormous range with the same number of bits**. Galaxy and atom fit in one format. The price is precision — the number of digits in the kernel is finite, so anything beyond them is **discarded by rounding**.

⚠ **"Computers are good at maths, so of course decimal arithmetic is exact"**

Plausible — you have never seen a calculator get it wrong. But the numbers floating point can hold are only **finitely many**, and a number it cannot hold is rounded to its nearest neighbour. Astonishingly, that includes 0.1 — 0.1 is an infinite fraction in binary (see the box below), so the "0.1" inside the machine is really a different number very close to 0.1. Small discrepancies can accumulate through a calculation. This is not a defect of the computer but a necessity of finite representation, and there are proper ways to handle it (chapter 41). One thing to remember for now: **floating point is not an exact number but a faithful approximation.**

∑ **Why 0.1 does not terminate in binary**

A fraction terminates only when its denominator is made solely of factors of the base. In decimal (base $10 = 2 \times 5$), $\frac{1}{10}$ ends after one digit. But binary's base is only 2, so $\frac{1}{10}$, with a 5 in the denominator, never terminates:

$$0.1_{10} = 0.0001100110011\ldots_2$$

It is the same as $\frac{1}{3}$ being the endless 0.333… in decimal. A finite significand has to cut this infinity off somewhere, and that cut is the source of the approximation.

6.3 Three incidents caused by approximation

Let us look ahead, numerically, at the faces "faithful approximation" wears in practice. (Shown here by calculation on the page; running it in C is chapter 41.)

Incident 1 — 0.1 + 0.2 ≠ 0.3. The most famous non-equation in the programming world. As we just saw, 0.1, 0.2 and 0.3 are all infinite in binary, so the machine holds each one's "nearest neighbour" instead. And the sum of 0.1's neighbour and 0.2's neighbour happens to round the **other way** from 0.3's neighbour. Written out in the 64-bit format —

- 0.1 + 0.2 in the machine = `0.30000000000000004440`…
- 0.3 in the machine = `0.29999999999999998889`…

Both are faithfully close to 0.3, but **they are not each other**. So when you ask "are they equal?" (`==`), the machine answers honestly: no.

Look at the internal representation and the whole incident is visible at a glance. The 64-bit format splits into [1 sign bit | 11 exponent bits | 52 significand bits], and writing these four numbers' 64 bits in hexadecimal —

number	64-bit internal representation (hex)
0.1	`3FB9 9999 9999 999A`
0.2	`3FC9 9999 9999 999A`
0.3	`3FD3 3333 3333 3333`
0.1 + 0.2	`3FD3 3333 3333 3334`

There are three layers to read here. First, the `9999`… in the significands of 0.1 and 0.2 is the repeating binary `0011` seen in hexadecimal, and the fact that **the last digit is `A` and not 9** is the

trace of rounding up while cutting the infinity off at 52 bits — the "cut" of the maths box is stamped right into the bits. Second, 0.3′s significand `3333`… is the same repetition at a different phase, and this one rounded down at the end. Third — the crux — 0.3 and 0.1+0.2 differ by **exactly one final bit**. They are the immediate neighbours, one tick apart (1 ulp, in the jargon). `==` answers "different" even to that one-bit difference, so floating-point "equality" breaks under a discrepancy of a single tick.

Incident 2 — so how do you compare? In the floating-point world the question of equality itself has to change — not "are they exactly equal?" but **"are they close enough?"** You settle on a tolerance (traditionally called **epsilon**, ε) and count the two numbers as equal if their difference falls inside it:

$$|a - b| < \varepsilon \quad (\text{e.g. } \varepsilon = 10^{-9})$$

That is the basic form; in practice there is one more layer — as numbers grow, so does the gap between neighbours (incident 3 below), so instead of a fixed ε it is safer to use a tolerance **proportional to the size of the numbers** (relative error). The concrete use of both, and their traps, is covered in C code in chapter 41. What to remember now is one sentence: **code that compares floating-point numbers with `==` is almost always suspect.**

Incident 3 — beside a large number, a small one disappears. That the significant digits are finite also means that when two numbers of very different sizes meet in one container, the smaller one **vanishes entirely**. Demonstrated with the 32-bit `float` (about 7 significant decimal digits) —

- The moment you store 8.000000001: the tail beyond 7 significant digits is cut and it simply becomes 8.0. It vanished **in the storing, before any addition**. The internal representation makes it starker — `float`'s 8.0 is `4100 0000`, and storing 8.000000001 gives the **same** `4100 0000`, because the nearest tick is 8.0. Two different numbers in decimal notation are one and the same number in bits.
- 8.0 + 0.00000008: the result is still 8.0, and the bits are still `4100 0000`. Why — compute the tick spacing (the distance between neighbours) of `float` at the size of 8.0 and you get $2^3 \times 2^{-23} = 2^{-20} \approx 0.00000095$. The 0.00000008 added does not reach even half a tick, so rounding leaves it where it was. This is called **absorption**.
- Conversely, **subtracting two nearly equal numbers**, as in $8.000001 - 8.0$, erases all the leading digits and leaves only the approximation in the last ones — a calculation that started with 7 significant digits produces an answer with one or two. This is called **cancellation**, and it is the chief culprit in eroding precision in numerical work.

The three incidents have a single root — the representable numbers are a **discrete set of ticks**, and the spacing between ticks widens as numbers grow. Near 0 the ticks are dense enough to distinguish something like 10^{-300}, but around 10^{16} the spacing exceeds 1 and **even integers get skipped**. It is no exaggeration to say that a feel for "how many digits can I trust?" (significant digits) is the whole of using floating point.

6.4 The chaos, and the contract called IEEE 754

The idea of floating point was old, but **how** to hold it — how many bits each for significand and exponent, which way to round — differed by company for a long time. IBM mainframes, DEC's VAX and Cray supercomputers each used a different format. The same program moved

to another machine gave **different answers**, and numerical programmers had to learn each machine's arithmetic habits afresh.

In 1985 a standard called IEEE 754 unified the chaos. It is a precise contract that pins down the format (1 sign bit + exponent + significand), the rounding rules, and the treatment of special values such as infinity and "not a number" (NaN). Today essentially every CPU and GPU follows this contract — the 32-bit format (C's `float`) and the 64-bit format (C's `double`) being the representatives.

The dates are worth noticing. The contract for floating point (IEEE 754, 1985) and the contract for the C language (ANSI C, 1989) were born within a few years of each other. It was an era in which the industry, unable to bear vendor-by-vendor arbitrariness any longer, turned to "let us write contracts" — and we meet this "age of contracts" again in chapter 10.

Q How do you choose between the 32-bit `float` and the 64-bit `double` in practice?

A The default is `double`. The 64-bit format handles about 15–16 significant decimal digits, which leaves room in most calculations. `float` (about 7 digits) is chosen for its size advantage where memory and bandwidth matter — large numbers of coordinates, graphics, machine learning. "double for precision, float for volume" is roughly right. Actual use in C is covered in chapter 41.

Q Integer overflow in chapter 5 and rounding in this chapter are both problems of "a finite container" — are they the same kind of problem?

A The root is the same, the symptoms differ. The integer container is finite in **range**, so it overflows at the end (and is perfectly exact until it does); the floating-point container is finite in **precision**, so it approximates a little everywhere (but its range is enormous). An exact but narrow container and a wide but approximating one — a feel for which container to use is a fundamental of handling numbers, and choosing types in C (chapters 23 and 41) is exactly that choosing.

To summarise this rung of the ladder: the decimal point is not in the bits but in an agreement. Pin the agreement down and you have fixed point; carry it in the data and you have floating point, whose agreement is unified by the contract IEEE 754. And numbers under that contract are not exact numbers but faithful approximations.

The next rung is characters. What has to be agreed in order to put "hello" in the lockers — the world of characters and text, and the story of the scars left where those agreements failed to match.

7 Characters and text - scars in the standard

By the end of this chapter

How to put letters into the lockers — that a character is, in the end, a number; what history that numbering table (the character code) went through on its way to Unicode and UTF-8;

and the scar that history left in C's syntax (trigraphs). For a reader who writes Korean this chapter is unusually vivid.

Looking back Chapter 6 said the decimal point is not in the bits but in an agreement. Then what about letters — is the shape "가" inside the bits?

A No; the principle is the same. Bits have neither shape nor sound. All there is is numbers, and there is a separate **agreed numbering table** saying which number is which letter. To store a letter is to store a number, and text is a sequence of numbers. This chapter is the story of those tables.

7.1 A character is a number

There is only one way to store "A" in a machine — make an agreement that "A shall be number 65" and store the number 65. Such a table of agreements, the correspondence between letters and numbers, is called a **character set** or character code.

The problem is that there were **several** such tables. Every company and every country made its own. The same number meant a different letter in different tables, and when two machines with different tables exchanged text the letters came out tangled. The rest of this chapter is the history of that tangle, and every turn of that history left a trace in today's C and computing.

7.2 ASCII and EBCDIC — two tables

Two branches of table established themselves in the 1960s. IBM's mainframe world used a table called **EBCDIC** — grown out of punched-card codes, so its layout looks strange today (the alphabet is not in consecutive numbers, among other things), but in IBM's world it was law.

On the other side, centred on the communications industry, came **ASCII**. Seven bits, that is a small table of 128 slots, holding the English upper and lower case, digits, punctuation, and the **control characters** that are invisible on screen. A control character is not a letter but an instruction to a device — "advance the paper one line", "sound the bell" — and their story comes in the next chapter (streams). And sitting at **slot 0** of this table is the **NUL character** whose face we learned in chapter 4. A character meaning "no character at all", which C later adopted as the mark for the end of a string (chapter 34).

ASCII became the winning table — nearly every character code today carries ASCII inside it. But 128 slots were tight even for English and nowhere near enough for the world's writing. Here the history of the scars begins.

7.3 ISO 646 — national variants and C's trigraphs

The first stopgap was **ISO 646**, the international edition of ASCII. The idea went: "keep one table, but allow each country to replace a few less important slots with its own letters." The slots designated for replacement happened to include symbols such as `[ ] { } \ | #`.

The result was a disaster for programmers. Open C code on a Danish terminal and Danish letters appeared where the braces should be. Same number, different table, different letter printed.

◉ **The ₩ sign — ISO 646's scar left in Korea**

Korea is a party to this history too. The Korean table (KS X 1003) replaced the backslash `\` with the won sign `₩`. On Korean computers of that era file paths looked like `C:₩Program Files₩...` — and, astonishingly, this scar can still be touched. Even on today's Korean Windows some fonts draw the backslash code as `₩`. Same number, same bits, and the table (the font) draws `₩`. This book's refrain, "what is stored and how it is read are separate things", is something Korean readers have witnessed on screen every day.

C was standardised in the middle of this confusion (chapter 10). Since C had to be usable in countries where braces were invisible, C89 provided a grotesque detour — the **trigraph**. Write `??<` and it was read as `{`; write `??/` and it was read as `\`; a three-character cipher. A scar of the character-code wars, carved into the grammar of the language.

Q Are trigraphs still used?

A No — and their end is a symbolic scene in recent C history. As the Unicode era arrived trigraphs lost their reason to exist and survived only as a trap in which a string like `??!` was unexpectedly transformed. C23 **removed** trigraphs from the standard. A scar carved in 1989 was erased only in 2023 — which also shows how much slower a standard is at **removing** something than at admitting it.

7.4 A hundred schools of eight bits, and Hangul

Once the byte settled at 8 bits (chapter 2), the **ISO 8859** family extended the table to 256 slots — the first 128 exactly ASCII, the second 128 holding the letters of one language region (Latin-1 for Western Europe, and so on). But since the upper 128 were used differently per region, the problem of tangled letters when you picked the wrong table remained.

And there were writing systems for which 256 slots were nowhere near enough. Hangul, Han characters, kana — East Asian scripts run to thousands and tens of thousands. The solution was multi-byte encodings using **several bytes per character** (Korea's EUC-KR among them). With lengths varying per character and methods varying per country, a Korean document read with the wrong table breaking into a parade of meaningless symbols — the so-called "broken characters" — was daily life for Korean computer users of that era.

7.5 Unicode and UTF-8 — one table, a clever way of holding it

In the end there was only one fundamental solution. **Put every letter in the world into one table.** That is **Unicode**, from the 1990s. Each letter gets a unique number (a code point), written like `U+AC00` (가). The table has more than a million slots.

The remaining problem was **how to hold** those large numbers in bytes. Spending 4 bytes on every letter is wasteful and, above all, incompatible with the mountains of existing ASCII text. The answer was the variable-length encoding **UTF-8** — letters in the ASCII range stay 1 byte as they are, and everything else takes 2–4 bytes. Thanks to the exquisite design by which an existing ASCII file is already valid UTF-8, UTF-8 has become effectively the only standard for the web and for source code today. It is what this book's examples and C23's `u8""` strings use as well (chapter 34).

∑ The byte structure of UTF-8

The length varies with the size of the code point. The leading bits of the first byte announce how many bytes this character has, and every continuation byte starts with `10`:

range	bytes	bit layout
U+0000 – U+007F	1	`0xxxxxxx` (= ASCII)
U+0080 – U+07FF	2	`110xxxxx 10xxxxxx`
U+0800 – U+FFFF	3	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000 – U+10FFFF	4	`11110xxx 10xxxxxx ×3`

Because continuation bytes always start with `10`, you can recover the character boundary even if you wake up in the middle of the text (self-synchronisation). Hangul syllables (around U+AC00) live in the 3-byte range.

⚠ "One character is one byte"

A misconception left over from the intuitions of the ASCII era, and especially dangerous when learning C — because C's `char` type is, despite the name, not a "character" but a **byte** (chapter 34). In UTF-8 the English `A` is 1 byte but the Hangul `가` is 3. The two letters of "안녕" are six bytes. Character count and byte count are different objects, and code in which the distinction has collapsed will certainly cause an accident when handling Hangul.

7.6 Same letter, several representations — the security terrain hidden in text

Unicode unifying the table did not end the problems. If anything, as the table grew and the representations layered up, new traps appeared. It is terrain any program handling text meets eventually, so let us survey it as a map. The root is single — **if the same thing can be written in two or more ways, the side checking it and the side interpreting it can disagree.**

① **Normalization — Hangul's NFC and NFD.** The letter "각" can be written two ways in Unicode: as one composed syllable (NFC, the single code point `U+AC01`) and as a sequence of jamo (NFD, the three code points `ㄱ` + `ㅏ` + `ㄱ`). They look identical, but the byte sequences are entirely different — so a naive byte comparison answers "각 ≠ 각". For Korean users this is not a textbook story: because macOS stored file names in the NFD family, it was common for Hangul file names crossing to another OS to appear with the jamo undone, or for names to go wrong inside archives. The solution is to **normalise to one form at the boundary** — and in a security context the order of that normalisation is decisive (see ④ below).

② **Overlong encodings — the back door UTF-8 once left open.** By the UTF-8 rules in the maths box above, `/` (U+002F) is the single byte `2F`. But early implementations generously accepted the same character written "long", in 2 bytes (`C0 AF`) or 3 — an **overlong** representation. From this came a classic attack: the security check looked only for `2F` to filter out path traversal (`../`), while the file system interpreted the overlong `C0 AF` as `/` too. A disagreement between checking and interpreting — that was the identity of the directory-traversal vulnerability that swept IIS servers in 2001. Today's standard therefore **declares overlong forms illegal and requires decoders to reject them**.

③ **The ghost of a vanished encoding — UTF-7.** UTF-7, made to send Unicode through 7-bit channels (early email), represents other characters using ordinary-looking ASCII letters (`+ADw-` becomes <). In the days when browsers guessed encodings automatically, attackers used this property to get through filters — a harmless-looking string at checking time that resurrects as a tag the moment the browser interprets it as UTF-7. UTF-7 was effectively retired because of these incidents, and today's norm has hardened into **do not guess the encoding; state it**.

④ **Traps in Unicode itself — homoglyphs and direction controls.** A large table also means there are **different letters that look identical**. Latin `a` and Cyrillic `а` are indistinguishable on screen — using this to build a fake domain that looks exactly like the real one is a **homograph attack** (which is why browsers show suspicious mixed-script domains back in their original notation). More cunning are the **directional control characters**: Unicode has invisible characters that reverse the display order of text, making it possible for **the order a human reads in the source and the order the compiler reads to differ** — "Trojan Source", published in 2021, used this technique to show that code looking perfectly fine to a reviewer could compile with different logic. Compilers began warning about such characters afterwards.

⑤ **When layers stack — escaping and multiple interpretation.** To display < as a letter on the web you write `<` (HTML escaping). Stack several such layers and accidents follow — decode once-escaped text in another layer and the original symbol resurrects; conversely, check but forget to escape and the input becomes code. The format-string vulnerability of chapter 19, SQL injection in databases and XSS on the web are all the same pattern — **making data be mistaken for the frame (the code)**. C's trigraphs were an ancestral case of the pattern: if `??/` happened to appear inside a string, the preprocessor turned it into a backslash, creating an escape the programmer never wrote (one of the reasons C23's removal of trigraphs is welcome).

> **Q** If the principle a programmer should take from these traps were reduced to one, what would it be?

> **A** **Do not let the representation change between checking and interpreting.** Unfolded into practical rules there are three — first, **do not guess the encoding; state it** (pin UTF-8 at the input/output boundary). Second, **normalise and decode first, and check afterwards** — reverse the order and the overlong and UTF-7 accidents come back. Third, **do not mix data into the frame** — a value always goes in the place for values (the `printf("%s", input)` idiom of chapter 19 is the smallest practice of this principle). All three come from this book's refrain that representation and interpretation are separate.

7.7 Sequences of letters — ways of holding a string

Now that we know how to hold one letter, a last question remains. When a **sequence** of letters — a **string** — goes into the lockers, how do we know "where does this string begin and end?" Chapter 3's refrain returns: memory does not know the boundaries of a lump; what knows is the agreement of the reader. There are several such agreements, so let us look at the main branches.

Method 1 — write the length in front (length-prefixed). Attach a number meaning "this many characters follow" in front of the string. The Pascal family used it, so it is also called a

Pascal string. Asking for the length needs no counting (read the number and you are done), and any byte at all — even character 0 — can be held as content. The price is that the size of the length field has to be fixed in advance — old Pascal held the length in one byte, so a string could not exceed 255 characters (chapter 2′s container story, again).

Method 2 — plant a marker at the end (NUL-terminated). Instead of writing the length, plant a special character at the **end** of the sequence — the NUL character of value 0 whose face we learned in chapter 4. **This is the method C chose.** Its virtue is extreme simplicity — you need only the starting point, and pointing anywhere in the middle of the sequence makes "from there to the NUL" a string. The price is threefold: to know the length you must **count all the way to the NUL**; you cannot hold a NUL in the content; and above all, forget to plant the marker and the reader runs on into other people's land. That last price is the terrain of more accidents than any other in C's history, and chapter 34 is the scene.

Method 3 — manage length and capacity together. The dynamic strings of modern languages usually manage three values as one bundle: [where the content is, the current length, the capacity of the container] — length queries are immediate, appending is free within the capacity, and when it runs out the content moves to a larger container. C++'s `std::string` and Rust's `String` have this structure, and proven's string handling, which we meet later, thinks in the same family by treating length explicitly (chapter 35).

◉ **The magic of fifteen characters — small string optimization in the MS STL**

Real implementations of method 3 have a clever double mechanism. In the Microsoft standard library implementation (MS STL) of neighbouring C++, `std::string` does not borrow a big warehouse (dynamic memory) for a short string — specifically **15 characters or fewer** (15 bytes plus the terminator) — but **simply holds it inside the body of the string object itself**. The technique is called SSO, small string optimization. Why 15 — because reusing, as a content buffer for short strings, the space of the pointer, length and capacity fields that must be in the object anyway comes out to exactly that size. The effect is explained by chapter 9′s knowledge: most real-world strings are short, and those now travel attached to the object with no warehouse round trip (allocation), moving together on the same cache line. Other implementations (the GCC and Clang standard libraries) use the same technique with different numbers — a modern textbook case of "the choice of representation is speed."

The terrain is wider still — representations for editors that manage a large document as a tree of pieces (ropes), views that point at [start, length] without copying the original, and so on: each use has its own fitting representation. One thing to remember: **the shape a string takes in memory is not one thing but a choice made by a language and a library, and that choice determines the character of its performance and safety.** What character C's choice (NUL termination) has, and how it must be handled, is faced head on in chapter 34.

> **Q** Why does a beginner need to know this messy history now? Is it not enough to use UTF-8 only?

A For anything new, UTF-8 alone is enough — that is today's practice and this book's premise. But the history is needed for two reasons. First, old encodings still flow through the world's files and systems, so when you meet broken characters you must be able to diagnose "ah, the wrong table." Second, C itself is a product of this history — the name `char`, NUL termination, the wreckage of trigraphs — so knowing the history makes all of C's strange corners read as wounds with a story. There is nothing to memorise. "A character is a number; there were many tables until Unicode unified them; and UTF-8 won as the way of holding them" — those three sentences are enough.

The next chapter is the last rung on the ladder of representation. Now that we can hold letters, it is the story of letters **flowing** — why a computer's input and output is shaped as "characters going past one line at a time", an answer that lies in the age of paper cards and typewriters.

Part III — The first program

Not yet translated: 13, 14, 15
(read these in the Korean edition)

Part IV — A minimal toolbox

Not yet translated: 16, 17, 18, 19
(read these in the Korean edition)

Part V — Declarations: how names are made

Not yet translated: 20, 21, 22
(read these in the Korean edition)

Part VI — Values and flow

Not yet translated: 23, 24, 25, 26, 27, 28, 29
(read these in the Korean edition)

Part VII — Memory

Not yet translated: 30, 31, 32, 33, 34, 35, 36, 37
(read these in the Korean edition)

Part VIII — The shape of data

Not yet translated: 38, 39, 40
(read these in the Korean edition)

Part IX — Precision

Not yet translated: 41, 42, 43
(read these in the Korean edition)

Part X — Structure

Not yet translated: 44, 45, 46, 47, 48
(read these in the Korean edition)

Part XI — Reading the standard library

Not yet translated: 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61
(read these in the Korean edition)

Part XII — proven — fundamentals, verified

Not yet translated: 62, 63, 64, 65, 66, 67, 68, 69, 70, 71
(read these in the Korean edition)

Part XIII — Closing

Not yet translated: 72, 73
(read these in the Korean edition)